A Platform for Validation and Verification of Models for Concrete and Concrete Structures

J.F. Unger, A. Robens-Radermacher, S.M. Rosenbusch & D.Tyagi
Department Safety of Structures, Division 7.7 Modeling and Simulation
Bundesanstalt für Materialforschung und -prüfung, Berlin, Germany

D. Iglezakis & M.Jafarkhani
Competence Center for Research Data Management
University of Stuttgart, Stuttgart, Germany.

ABSTRACT: The reliable validation and verification of simulation models is a key challenge in computational engineering, especially for complex materials and multi-physics problems. Existing approaches are often computationally intensive and tailored to specific setups, making generalization and objective comparison across models and codes difficult. This paper presents a concept for a reproducible and extensible benchmarking platform designed for any FEM or CFD-based simulation setup, with concrete modeling as a motivating example.
The platform aims to facilitate data exchange between experimentalists and simulation researchers, enable transparent comparison of models and implementations, and support referencing models with quality criteria in engineering standards. Key components include a machine-readable database structure for experimental metadata, standardized calibration procedures, and quality metrics that account for uncertainties. The platform supports integration of open-source constitutive model libraries into commercial codes, promoting interoperability and community-driven benchmarking. A federated database infrastructure enables sharing and publication of experimental and simulation results, with full provenance tracking to ensure reproducibility. Interactive visualization tools allow users to query, explore, and compare models and results.
While the conceptual framework is well developed, a complete implementation is still in progress. Community contributions are needed to realize a broad set of benchmarks and develop tool-agnostic, machine-readable model definitions. The proposed platform aims to advance reproducibility, transparency, and robustness in computational modeling and simulation practice across engineering domains.

1 MOTIVATION

In civil engineering, particularly for concrete structures, a significant gap persists between advanced academic research in computational mechanics and its application in practice. The academic community continues to develop highly sophisticated models that capture complex phenomena such as cracking, creep, shrinkage, and reinforcement interaction. However, these models are often not validated for practical use, limiting their integration into engineering workflows. Conversely, practitioners typically rely on well-established design codes and safety factors—an approach that is understandable given the high stakes of structural engineering. In some cases, advanced methods are adopted without comprehensive validation, raising concerns about reliability and regulatory acceptance. This situation reflects a missed opportunity: advanced simulation capabilities have the potential to enable more efficient, sustainable, and innovative designs. Yet, their adoption remains slow due to concerns about trustworthiness, standardization, and compliance. The reluctance of the construction industry is reasonable, as concrete structures such as bridges,

tunnels, and high-rise buildings are typically built only once, making the consequences of modeling errors severe. Current design standards, while proven safe, often lead to conservative designs that increase material consumption and costs. Bridging this gap requires stronger collaboration between academia and practice. By jointly defining validation standards and aligning research developments with practical needs, both communities can unlock the full potential of advanced computational methods for safer, more sustainable, and cost-effective structures.

However, for the mechanics community to meaningfully influence the development of future engineering standards and enable the adoption of advanced simulation methods in practice, several critical challenges must be addressed systematically:

- **Formal model definitions:** Current (constitutive) models for concrete exist primarily as academic publications or software-specific implementations. To influence standards development, we need software-independent, mathematically rigorous machine readable definitions — including their scope of applicability, parameter identification procedures, and uncertainty bounds. Without such formalization, regulatory bodies cannot assess or approve these models for standard inclusion. However, even with rigorous formal definitions, the specifications will not be unique and complete in all mathematical details, making verification benchmarks essential to test whether two different model implementations are equivalent and produce consistent results.

- **Structured validation data:** The mechanics, structural engineering, and experimental communities must jointly define the experimental data required for model validation and agree on how it should be structured and documented. Rather than relying on ad-hoc experimental campaigns, we need systematic frameworks that specify the types of tests, measurement protocols, and metadata required to validate different aspects of concrete behavior under various loading and environmental conditions.

- **Software verification:** Industrial adoption requires confidence in software implementations. The lack of standardized, publicly available benchmark problems with either analytical or high-quality numerical solutions hinders the verification of commercial and research codes. Transparent, reproducible benchmarking infrastructure is essential for detecting implementation errors and building trust in simulation tools among practitioners and regulatory bodies. Additionally, such benchmarks enable testing whether two models that appear formally identical according to their mathematical definitions are actually implemented identically and produce equivalent results.

- **Model calibration and uncertainty quantification:** Concrete models typically involve numerous parameters that must be calibrated against experimental data, yet current approaches often lack systematic frameworks for parameter identification and uncertainty propagation. For standards inclusion, models require robust calibration procedures that account for experimental uncertainties, parameter correlations, and model form uncertainty. This includes establishing standardized protocols for parameter estimation, sensitivity analysis, and uncertainty propagation from material properties to structural response predictions.

- **Model validation:** Academic models are often validated against limited experimental datasets under controlled laboratory conditions. For inclusion in engineering standards, models must demonstrate robust performance across diverse real-world scenarios, with clear uncertainty quantification and well-defined limitations. This requires systematic validation campaigns that bridge the gap between laboratory experiments and field applications. Crucially, these validation results must be made publicly available through standardized platforms, similar to verification benchmarks, enabling transparent assessment of model performance and facilitating regulatory review for potential standard inclusion.

This article presents a concept procedure designed to address these challenges systematically, enabling the mechanics community to contribute actively to the evolution of engineering standards for concrete structures. Rather than proposing immediate solutions, it maps the technical infrastructure and methodological requirements necessary to bridge the gap between academic innovation and industrial practice, ultimately facilitating the responsible integration of advanced simulation capabilities into structural engineering standards.

2 INTRODUCTION

Advanced simulations increasingly support safetycritical decisions in structural engineering and infrastructure management, yet the formal specification of the underlying mathematical and constitutive models remains fragmented and tool-dependent. Input decks of proprietary solvers often serve as the only effective "model definition", while academic publications provide human-readable but not machine-actionable formulations of PDEs, boundary/initial conditions, and constitutive laws. This lack of a software-independent representation impedes: (i) transparency (what was actually simulated), (ii) interoperability (reuse across tools), and (iii) verifiability (can two implementations be shown equivalent beyond superficial similarity).

Emerging efforts begin to close this gap. The VMAP initiative[1] is extending its result-exchange focus toward model descriptions. Ontology-driven approaches such as MathModDB[2] and MathAlgoDB[3] within the NFDI initiative MaRDI aim to formalize mathematical objects (PDE operators, domains, tasks) for automated interpretation, though the lack of advanced CAD models to support the description of geometries still limits direct application to complex engineering use cases. In parallel, BIM (IFC) models enriched with FEM metadata ((Rudenko and Petryna 2025)) and neutral CAD/geometry standards such as STEP offer a path to generate simulation-ready representations with embedded semantics (materials, boundary conditions, coordinate systems). The most critical challenge remains the constitutive model definitions. There are efforts to close this gap. MFront is a toolbox for defining constitutive models in a simulation tool agnostic way using a domain specific language, with interfaces to multiple solvers (Helfer et al. 2020). Other domains have developed structured model definition languages, such as Modelica[4], which enable machine-readable model specifications. However, these languages are often closely linked to particular solver ecosystems and have not seen widespread adoption in structural engineering. For describing individual equations in a machine-readable format, standards like MathML[5] and OpenMath[6] offer XML-based representations. Nevertheless, extending these approaches to fully capture complex constitutive models and their algorithmic implementations remains a significant challenge.

Leveraging these structured sources for benchmark definitions would allow a single machine-readable problem specification to be interpreted by multiple solvers via thin adapters, avoiding repeated manual encoding. This is directly relevant for verification benchmarks, e.g. community sets inspired by (NAFEMS 1993) or for CFD suites[7] or Evadat [8].

Model verification is a necessary prerequisite for meaningful validation. Functional equivalence of a referenced model can only be established when two independent implementations produce consistent results on agreed-upon verification benchmarks. Formal, reusable benchmark specifications therefore enable (i) tool-to-tool comparability, (ii) detection of hidden implementation deviations, and (iii) the possibility to reference a constitutive or structural model in standards with confidence that independently maintained codes behave consistently. This reduces today's de-facto "black box trust" and lowers the barrier for quality assurance (future certification could become procedural rather than bespoke).

Validation addresses the ability of the model to correctly represent the physics itself. It is inherently hierarchical: widely accepted components (e.g. linear elasticity under small strain) form a base of reusable evidence; higher layers require case-specific validation when for specific geometries, load configuration, multi-physics models, specific constitutive models, or when other phenomena such as rate effects or evolution of material properties due to hydration, shrinkage etc. are introduced. Data-driven or model-free constitutive approaches ((Stöcker et al. 2024)) amplify this need: without broad, structured datasets, generalization claims remain untestable.

Engineering lacks the mature data/model ecosystems seen in machine learning, such as huggingface[9], kaggle[10], and roboflow[11]. In engineering domains, only a few domain-specific platforms exist, such as for alkali–silica reaction (Saouma et al. 2021). The SoilModels platform[12] offers downloadable implementations and

[1] www.vmap-standard.org
[2] www.mathmoddb.org
[3] www.mathalgodb.org
[4] https://modelica.org/
[5] https://www.w3.org/TR/mathml4/
[6] https://openmath.org/
[7] https://wwwold.mathematik.tu-dortmund.de/~featflow/en/benchmarks/cfdbenchmarking.html
[8] https://evadat.com
[9] https://huggingface.co/
[10] https://www.kaggle.com/
[11] https://roboflow.com/
[12] https://soilmodels.com/

datasets for validating constitutive models in geomechanics. The LEAP project (Liquefaction Experiments and Analysis Projects) provides structured databases for validating numerical procedures in soil liquefaction analysis (Vasko et al. 2018). Concrete creep data is provided in (Hubler et al. 2015). Other initiatives, such as NorSand4AI, offer synthetic datasets for triaxial test simulations to support data-driven constitutive modeling (Ozelim et al. 2024).

To scale validation, the community requires open, FAIR, machine-readable validation datasets capturing: test setup geometry and coordinate systems, sensor definitions (type, position, orientation, unit, sampling), boundary/loading history, material provenance and uncertainty, and processed plus raw signals. Structured linkage between these datasets and formal model definitions (above) enables (i) shared calibration workflows (e.g. Bayesian procedures), (ii) reuse of validation evidence across near-identical cases, and (iii) objective comparison of traditional versus data-driven constitutive formulations.

In summary, a vendor-agnostic, semantically explicit representation of simulation problems and models—paired with structured, provenance-rich validation data—is essential for reproducibility, trust, and eventual incorporation of advanced models into engineering standards. Community-driven, open infrastructure is required; isolated proprietary formats risk further fragmentation. The platform concept presented here addresses these gaps by combining machine-readable benchmark specifications, automated provenance capture, ROCrate-based packaging, and queryable publication to support verification, validation, and transparent comparison across tools.

3 GENERAL CONCEPT

3.1 *Prerequisites and Requirements*

A robust benchmarking concept for computational engineering must address several foundational requirements to ensure transparency, reproducibility, and community adoption. First, the choice between a central database and individual datasets with a shared structure and metadata (ROCrates[13]) impacts both discoverability and flexibility. A central database enables unified search and access, but requires ongoing maintenance and curation. It is also often less flexible in the way it can accommodate diverse data types and evolving standards. In contrast, individual ROCrates allow decentralized contributions and versioning, though they require a central registration platform to simplify discovery. Intellectual property rights and selective access are critical, requiring mechanisms for proper citation, licensing, and controlled sharing of data and results. A central database must create complex access control mechanisms to achieve that, whereas ROCrates enable a user to define the access rights on a per-ROCrate basis. In addition, the authorship of individual ROCrates is clearly defined allowing for a proper citation of individual contributions, which is often more complex to achieve in central databases.

[13] `https://www.researchobject.org/ro-crate/`

3.2 *Potential queries and use cases*

In order to illustrate the scope of the platform, potential questions that should be answered by querying the database are listed below:

1. Who (name and organization) has published results for the simulation tool FEniCSx and which benchmarks are implemented? This might be of interest for a user who wants to compare results for a specific tool and check if the benchmark of interest is implemented. Also enables a selection on which ROCrate producer you trust, e.g. only from the developers of a tool, or the opposite, a third-party provider that has no affiliation with the tool.

2. Which benchmarks in the Mechanics domain use a Drucker-Prager constitutive model? This might be of interest for a user that wants to compare results for a specific material model and compare its implementation in different tools.

3. For which software tools results with higher-order elements are published? What order is implemented? A user might be interested to identify tools that support specific features, e.g. higher order elements.

4. What is the sensor position in a global coordinate system for the displacement sensor measuring the deformation at the top right corner of the specimen for the plate with a hole test? This might be of interest when a developer implements a new benchmark and needs all the required metadata in a machine readible way.

5. What is the order of convergence in a specific benchmark (e.g. plate with a hole) for

the maximum von Mises stress and the computed displacements in y-direction at the right corner of the specimen for all the implemented tools using the newest version? Compare that with the analytical solution and plot for an h-refinement of the mesh with linear isogeometric elements the corresponding error for stresses and deformations. This is interesting to check if the implementation of the benchmark is correct and if the tools behave as expected and yield comparable results.

6. Which metrices have changed when moving from version 0.8.0 to the latest version of FEniCSX? What is that version? This might be interesing for users who want to change to a newly published software version and ensure consistency of results.

7. Are there any results in the database that have similar inputs (parameters, tool with version and same benchmark) as a locally provided ROCreate? If yes, compare the results and visualize the differences. (This is e.g. interesting in a local CI-pipeline to check if the local results are consistent with already published results.)

4 SUMMARY OF THE PLATFORM CONCEPT

Figure 1 shows an overview about the interacting components of the benchmarking platform that is discussed in more detail in the next sections. The git repository comprises all the relevant source code, including the actual benchmarks implemented into a workflow system, the documentation that is automatically built and hosted with ReadtheDocs, and a Jupyter notebook for post-processing and visualization of results for each benchmark. The latter serves as a basis for potential users and should only serve as examples on potential user questions.

The workflows are executed on an arbitrary server. Thus, users can run only specific benchmarks of interest on their own infrastructure with only a subset of the tools implemented in the repository - requiring acces to only those specific license files in case of proprietary software.

4.1 *Provenance*

An important aspect is the provenance of a benchmark execution performed for a specific tool and version. It is not sufficient to just publish some result metrics (e.g. simulated deformations at a specific point, or integral measures such as the L2 error compared to the analytical solution), but the full provenance of how these results were obtained must be captured. This includes all input files (or intermediate files e.g. a mesh file if the meshing process is part of the benchmark), the exact software version (including dependencies i.e. the full conda environment or a dockerfile). This provenance information is essential to ensure reproducibility and to allow other users to audit the results. In our approach, we propose to use workflow systems (e.g. Snakemake[14], Nextflow[15], or CWL-based systems[16]) that automatically capture the full provenance of a workflow execution without requiring manual user interaction. The output of the workflow is then an ROCrate that encapsulates all relevant information about the benchmark execution, including input data, output data, the workflow definition, and the provenance record. In the current implementation, we are developing a snakemake reporter plugin [17] that captures this provenance information based on the Provenance Run Create profile [18] discussed in (Leo et al. 2024).

5 BENCHMARK DEFINITION

A benchmark in the sense of the platform is a simulation problem, primarily in the domain of the numerical solution of partial differential equations (PDEs). In the application case of verification, an analytical solution is available to compare the numerical results against. In the application case of validation, experimental data is available for comparison. In both cases, a benchmark is defined by a set of input parameters and output quantities. In the current implementation, we only focus on computational accuracy, i.e. performance aspects such as computation time and memory consumption are of minor importance, even though the information is captured in the provenance record of the workflow execution.

5.1 *Inputs*

The set of inputs is specific to each benchmark. The following are examples of input types:

[14] `https://snakemake.readthedocs.io/`
[15] `https://www.nextflow.io/`
[16] `https://www.commonwl.org/`
[17] `https://github.com/izus-fokus/snakemake-report-plugin-metadata4ing`
[18] `https://www.researchobject.org/workflow-run-crate/profiles/`

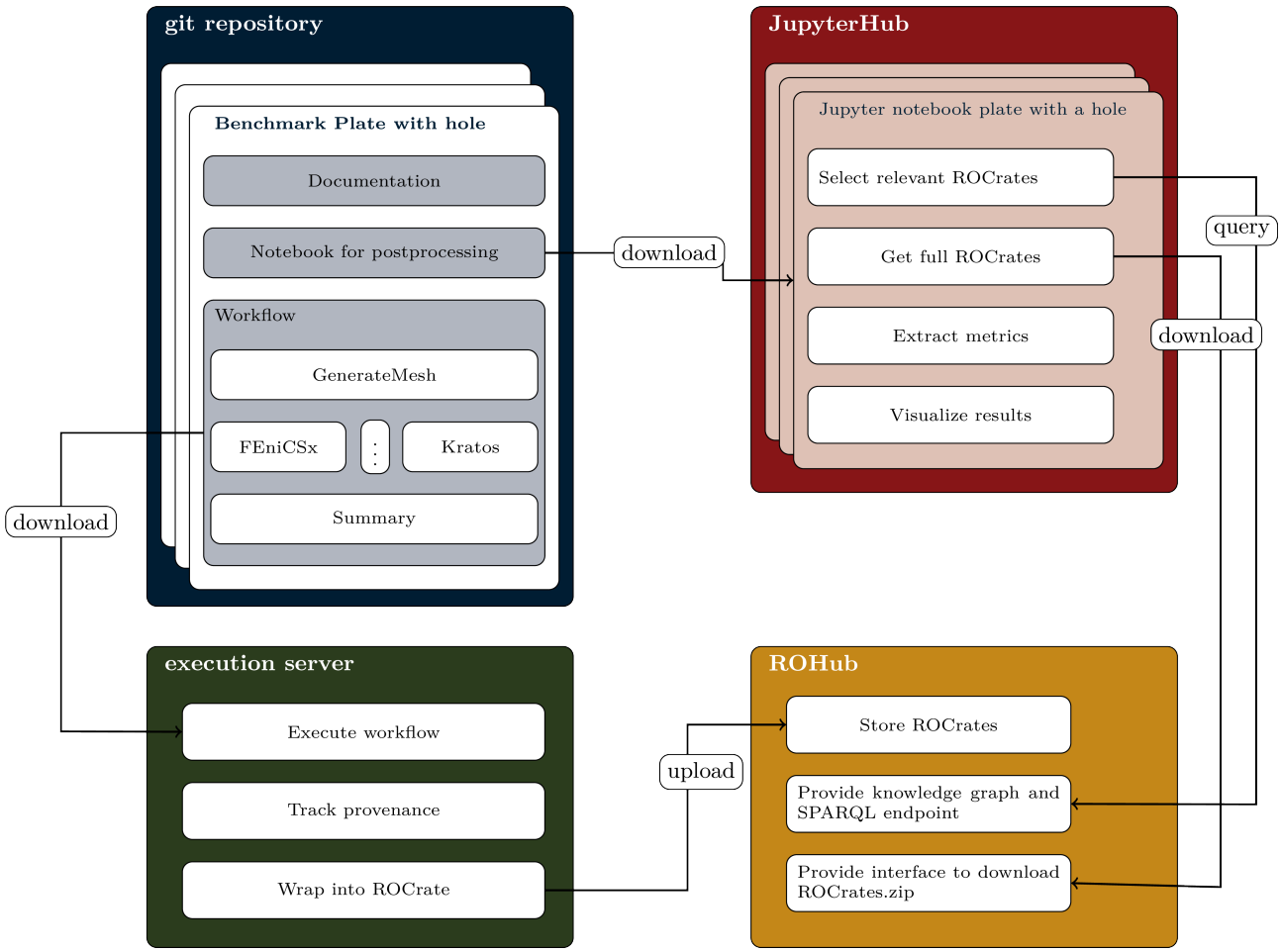

Figure 1: Overview about the interacting components of the benchmarking platform.

- Geometrical quantities: Information about the geometry of the model, including dimensions and shapes.

- Meshing properties: Details about the finite element mesh, such as element order and element density. In simple cases, the mesh generation is automated and for each configuration of e.g. mesh density and element order, a mesh is generated and then used for all simulation tools. In more complex cases, the mesh is directly provided as an input file.

- Material models: Defines the constitutive behavior of the materials used in the simulation. This includes the mathematical formulation of the model as well as any specific parameters required for its implementation. In the current setup, we assume that the material model is implemented in each simulation tool, and the benchmark input only specifies which material model to use (e.g. linear elastic, Drucker-Prager, etc.). In future extensions, we plan to allow the material model definition to be provided in a machine-readable format (see the section tool-agnostic model description).

- Material parameters: Specific properties of the materials, such as density, Young's modulus, and Poisson's ratio, specific to each material model.

- Initial conditions: The state of the system at the beginning of the simulation, including displacement and velocity fields. In a similar way, these fields should be defined using the tool-agnostic mathematical description language.

- Boundary conditions: Constraints applied to the model boundaries, such as fixed supports or applied loads.

- Sensor definitions: Locations and types of sensors used to measure specific quantities during the simulation, e.g. displacement at a specific point, stress at a specific location, L_2-norm of an error or reaction forces integrated over a (sub-)boundary. In the current implementation, these quantities are hard-coded, but we aim to develop an ontology to

define these sensors in a machine-readable way.

- Coordinate systems: Definition of the coordinate system used in the simulation, e.g. Cartesian, cylindrical, spherical. If no local coordinate systems is provided, a global cartesian coordinate system is assumed.

It is important that all parameters are defined in a machine-readable way, including their physical units. This is crucial to ensure that different simulation tools interpret the inputs consistently and produce comparable results. Any numerical parameter is provide in json with a value and it unit, e.g.

```
"length": {
    "value": 1.0,
    "unit": "m"
    },
"load": {
    "value": 100.0,
    "unit": "MPa"
    },
```

In our implementation, the `pint` library in python is used to handle the units [19] and perform any necessary conversions, so the units have to be compatible. Please note that in the provenance knowledge graph that is used for tracking parameters and results, units are automatically converted to QUDT units[20].

5.2 *Outputs and metrics*

The output of a benchmark generally includes raw data produced by the simulation tools, such as displacement fields, stress fields, reaction forces, and other relevant quantities. These are often stored in tool specific binary formats, which are supposed to be stored in a summary.zip file within the ROCrate (per tool and per benchmark). For visualization purposes, it is recommended to store the time series field data (e.g. stresses, displacements) in a common format such as VTK or XDMF that does not require a license to the specific simulation tool. In order to allow a comparison of results across different tools, a common set of result metrics is defined in the benchmark inputs based on the sensor concept. These can be scalar or vector quantities, point values or integrated values over a (sub-)domain and in particular time series data. Examples are the displacement at a specific point, the maximum von Mises stress in the domain, or the reaction force integrated over a boundary. Some sensors might need to reference the analytical solution to compute an error, e.g. for a convergence study.

5.3 *Compute environment*

One of the key challenges when comparing results from different simulation tools is ensuring that the compute environment is consistent, reproducible by others in the future, and well-documented. This includes the version of the software tool used to solve the benchmark as well as potential dependencies such as libraries or compilers. This can be done using containerization technologies such as Docker or Singularity, which encapsulate the software and its dependencies in a portable format. Alternatively, environment management tools like Conda can be used to create isolated environments with specific versions of software and libraries. It is crucial that the compute environment is documented in a way that allows others to recreate it exactly, ensuring the reproducibility of benchmark results.

6 ROCRATES FOR PACKAGING BENCHMARK RESULTS

ROCrates[21] provide a standardized way to package and share benchmark results, including all relevant metadata, input parameters, and output data. It is essentially a zip file that contains all the necessary files and a metadata file in JSON-LD format that describes the contents of the package. The metadata includes information about the benchmark, such as its name, description, authorship, version, and license, as well as details about the input parameters and output data. In general, an ROCrate can directly contain the information, but it can also reference external resources, such as datasets or software tools, using persistent identifiers (e.g. DOIs). This allows for a more modular approach to packaging benchmark results, where different components can be reused and shared across different ROCrates. Writing the metadata file can be done manually, but it is often more efficient to use tools that automate the process based on predefined templates or schemas. In our implementation, we use the snakemake reporter plugin to generate the metadata file automatically based on the workflow definition and the results produced by the benchmark, so the developer of

[19] `pint`

[20] `https://qudt.org`

[21] `https://www.researchobject.org/ro-crate/`

a benchmark or a tool does not need to manually write complex RDF or JSON-LD metadata. An alternative is the module `rocrate`[22] that provides a high-level API to create and manipulate ROCrates in python.

7 ROHUB FOR PUBLISHING AND QUERYING BENCHMARK RESULTS

ROhub[23] is a platform for publishing, sharing, and discovering ROCrates. It provides a web interface for browsing and searching ROCrates, as well as an API for programmatic access. ROHub also supports versioning and citation of ROCrates, making it easy to track changes and give credit to authors. In addition, ROHub provides a SPARQL endpoint that allows users to query the metadata of ROCrates using the SPARQL query language. This enables users to search for specific benchmarks based on criteria such as author, date, keywords, or input parameters. In our implementation, we aim to use ROHub as the central repository for publishing benchmark results, allowing users to easily discover and access the results of different benchmarks (or simulation tools with their respective versions).

8 JUPYTER NOTEBOOKS FOR POSTPROCESSING AND VISUALIZATION

Jupyter notebooks are used for postprocessing and visualization of benchmark results. They provide an interactive environment for analyzing the output data, generating plots, and extracting relevant metrics. Each benchmark is accompanied by a Jupyter notebook that serves as a template for users to customize their analysis. The notebooks can query the ROHub SPARQL endpoint to retrieve the relevant ROCrates, download the full ROCrates, and perform the necessary postprocessing steps. This allows users to easily explore the results of different benchmarks and gain insights into the performance of various simulation tools.

9 SUMMARY AND OUTLOOK

This article outlines a concept for a reproducible and extensible benchmarking platform for computational engineering, with a focus on the numerical solution of partial differential equations (PDEs). The proposed platform is intended to facilitate the comparison of different simulation tools and their implementations by providing a standardized way to define, execute, and share benchmarks. Key elements of the concept include machine-readable benchmark definitions, ROCrates for packaging results, ROHub for publishing and querying, and Jupyter notebooks for postprocessing and visualization.

At present, this work represents a conceptual framework rather than a fully implemented solution. Many aspects remain to be developed, and the realization of the platform will require substantial input and collaboration from the community. Contributions are especially needed for the creation of benchmarks, tool adapters, workflows, and the development of machine-readable, tool-agnostic model definitions. Any feedback, suggestions, or participation from interested researchers and practitioners is highly appreciated to help shape and advance this initiative.

[22] `https://pypi.org/project/rocrate/`

[23] `https://rohub.org/`

REFERENCES

Helfer, T., J. Bleyer, T. Frondelius, I. Yashchuk, T. Nagel, and D. Naumov (2020). The mfrontgenericinterfacesupport project. *Journal of Open Source Software 5*(48), 2003.

Hubler, M. H., R. Wendner, and Z. P. Bažant (2015). Comprehensive database for concrete creep and shrinkage: Analysis and recommendations for testing and recording. *ACI Materials Journal 112*, 547–558.

Leo, S., M. R. Crusoe, L. Rodríguez-Navas, R. Sirvent, A. Kanitz, P. De Geest, R. Wittner, L. Pireddu, D. Garijo, J. M. Fernández, I. Colonnelli, M. Gallo, T. Ohta, H. Suetake, S. Capella-Gutierrez, R. de Wit, B. P. Kinoshita, and S. Soiland-Reyes (2024). Recording provenance of workflow runs with ro-crate. *PLoS ONE 19*(9), e0309210.

NAFEMS (1993). *Selected Benchmarks for Material Nonlinearity – Volume 1*. NAFEMS Ltd., East Kilbride, UK. NAFEMS Benchmark Report R0026.

Ozelim, L. C. d. S. M., M. D. T. Casagrande, and A. L. B. Cavalcante (2024). Norsand4ai: A comprehensive triaxial test simulation database for norsand constitutive model materials. *Geoscientific Model Development 17*, 3175–3197.

Rudenko, I. and Y. Petryna (2025). An approach to automatic bim-based generation of fe models of different complexity and fe dimensionality. *Buildings 15*(2), 171.

Saouma, V., A. Sellier, S. Multon, and Y. Le Pape (2021). Benchmark problems for aar fea code validation. In V. Saouma (Ed.), *Diagnosis & Prognosis of AAR Affected Structures*, Volume 31 of *RILEM State-of-the-Art Reports*. Springer, Cham.

Stöcker, J., S. Heinzig, A. Khedkar, and M. Kaliske (2024). Data-driven computational mechanics: comparison of model-free and model-based methods in constitutive modeling. *Archive of Applied Mechanics 94*, 2683–2718.

Vasko, A., M. ElGhoraiby, and M. Manzari (2018). Leap-gwu-2015 laboratory tests. DesignSafe-CI.